

Software Requirements and Design Document

For

Group 28

Version 1.0

Authors:

Brooklyn A

Carly C

Tomas C

Philip Q

Brandon M

1. Overview (5 points)

Byte2Byte is a restaurant inventory management system designed to integrate with Square point of sale. The system automatically links every sale processed through Square to the specific ingredients required for that menu item. Being able to track ingredient usage in real time allows restaurant managers to be up to date on their inventory, which in turn leads to reduced waste, prevented shortages, and overall better managed costs.

In addition to inventory tracking, Byte2Byte calculates the cost of goods sold (COGS) for each item based on the ingredients portions and costs. That allows it to generate daily reports comparing sales revenue with COGS as well as providing low-stock notification to managers on ingredients. Unlike certain existing solutions that are expensive and geared towards larger businesses, Byte2Byte is intended to be an open-source, cost-effective, and accessible platform.

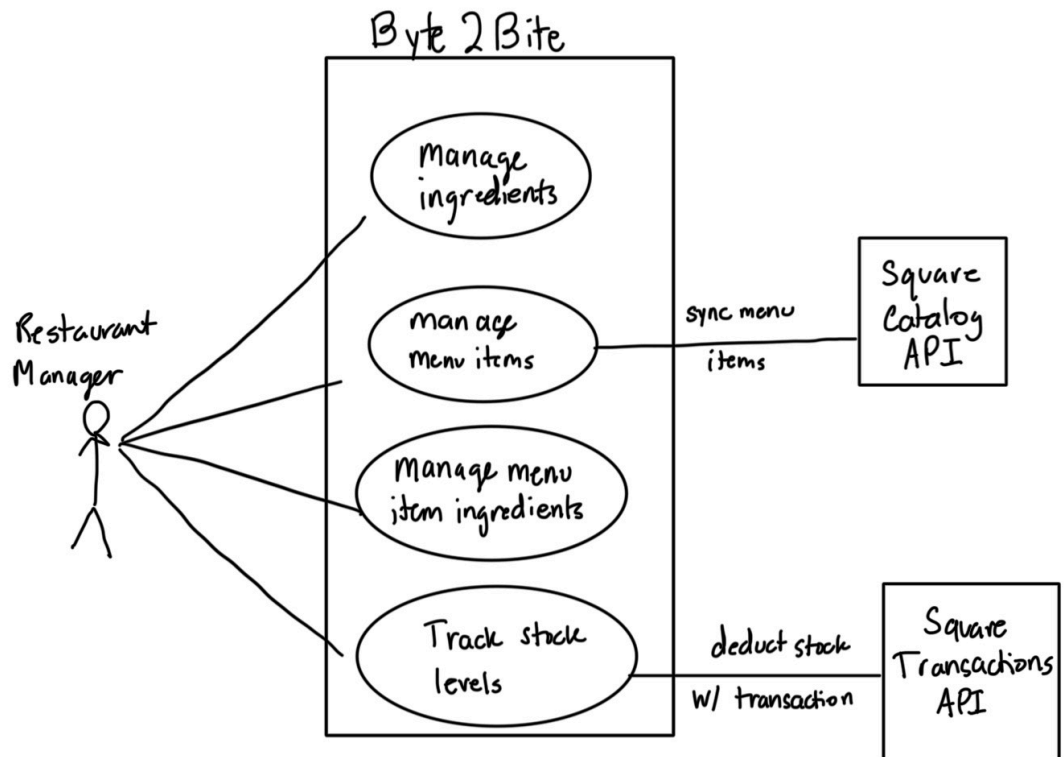
2. Functional Requirements (10 points)

1. Square Catalog Syncing, Priority: High
 - a. The system shall retrieve the complete set of catalog items from the connected Square account and create corresponding item records in B2B, including item names, IDs, categories, and current active/inactive status.
Rationale: establishes a one-to-one mapping between Square catalog items and B2B items.
2. Catalog Change Detection, Priority: High
 - a. The system needs to detect when catalog items are added, modified, or removed in Square and shall update the corresponding B2B records so that item metadata remains consistent with Square.
3. Square Transaction Logging, Priority: High
 - a. The system needs to log all transactions, specifically the line items from them and the quantity sold in order to deduct the ingredient's stock associated with the menu item.
4. Ingredient Master Data, Priority: High
 - a. The system must allow creation and maintenance of ingredient records with unique name, unit of measure, current quantity in stock, low-stock threshold etc.
5. Inventory Deduction on Sale, Priority: High
 - a. Upon ingestion of a sale for a trackable catalog item, the system shall deduct from on-hand inventory the bill of materials quantities multiplied by the sold quantity for each ingredient.

3. Non-functional Requirements (10 points)

1. Performance & Reliability – High Priority
 - a. The system shall process sales and update inventory within 2 seconds of a Square transaction and maintain at least 99.9% uptime.
2. Security – High Priority
 - a. All data shall be transmitted over HTTPS, with secure storage of credentials and tokens following industry best practices.
3. Scalability & Maintainability – Medium Priority
 - a. The system shall handle at least 1,000 daily transactions, be designed with modular code, and allow for straightforward updates or integration with other POS systems.

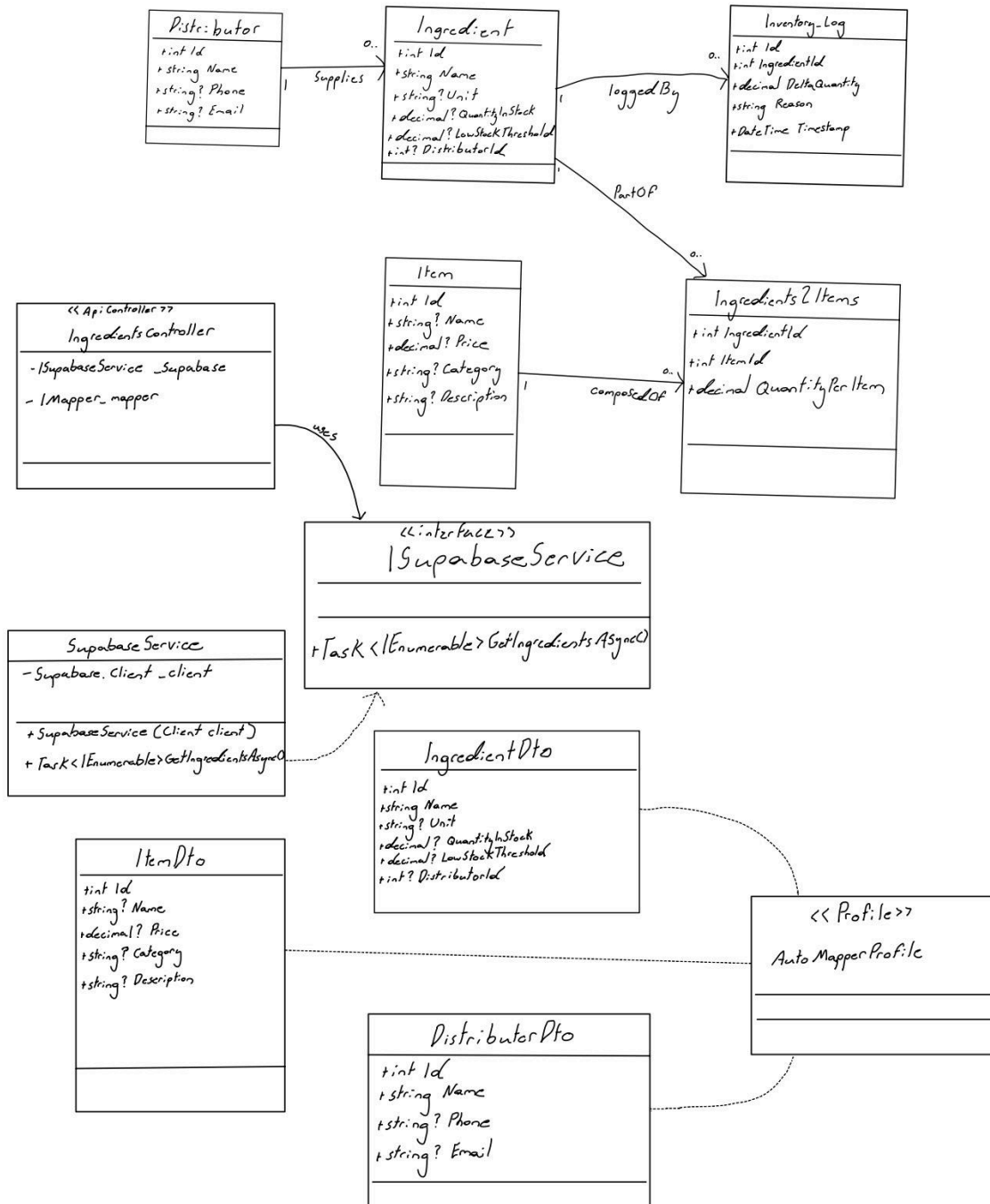
4. Use Case Diagram (10 points)



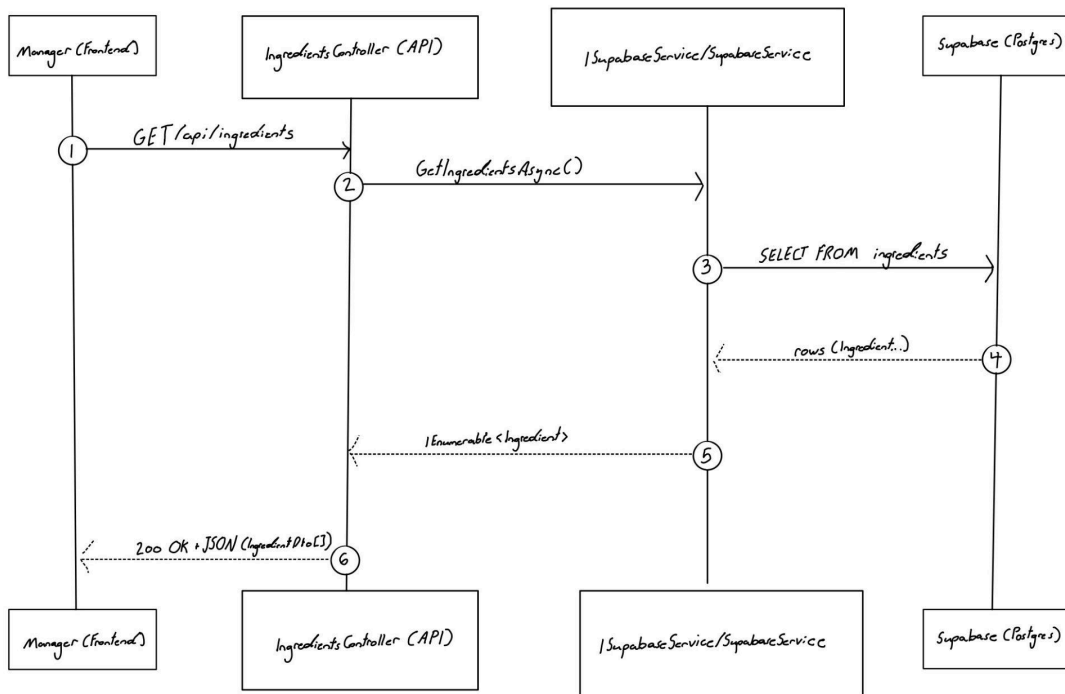
5. Class Diagram and/or Sequence Diagrams (15 points)

Class Diagram

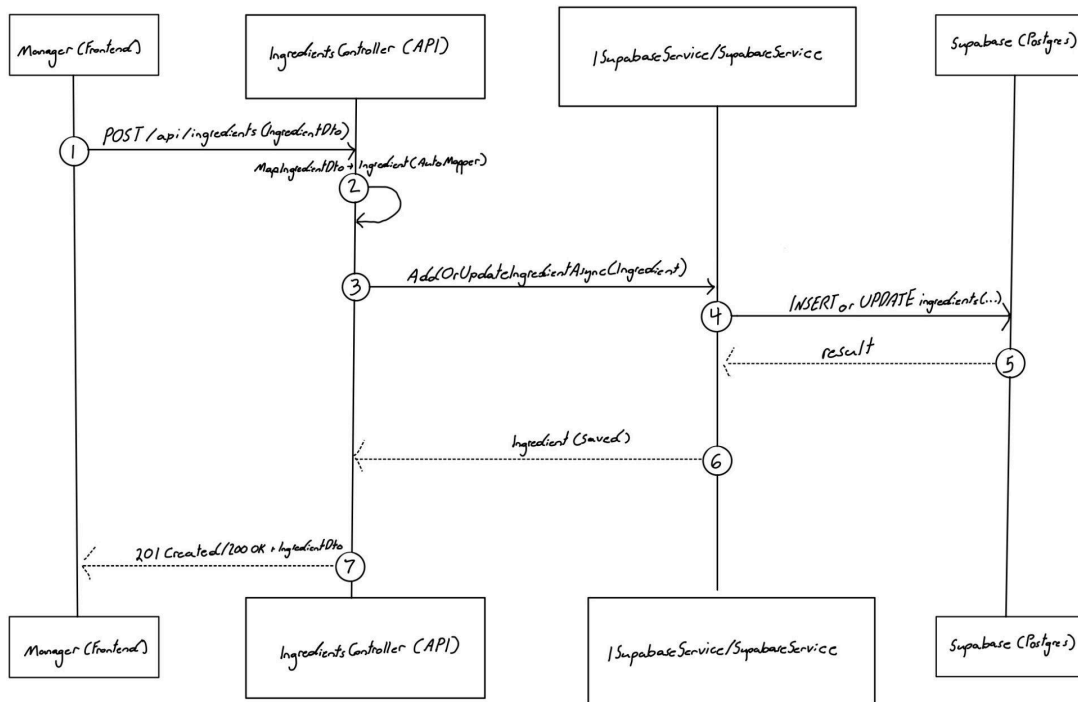
Class Diagram



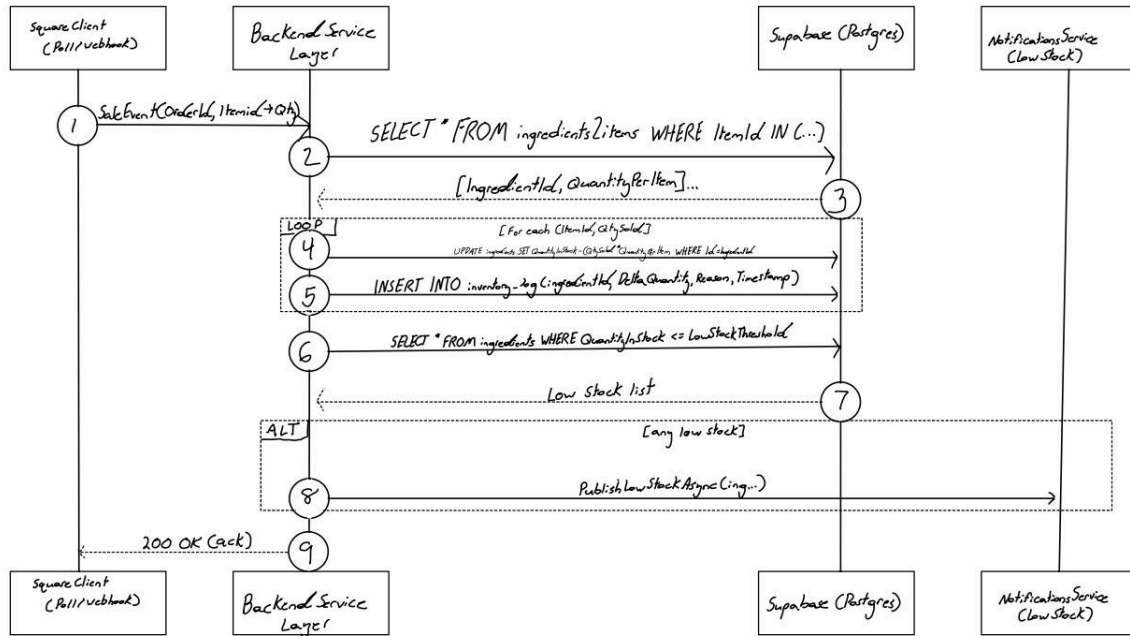
Use Case #1:



Use Case #2:



Use Case #3



6. Operating Environment (5 points)

Byte2Bite will operate as a web based application, accessible through browsers. The frontend will run on React/TypeScript while the backend will run on a .NET 9 environment hosted on a cloud platform.

The database is powered by Supabase, which will provide secure and scalable storage. The system will coexist with Square API, ensuring real-time synchronization and catalog items.

Restaurant managers will be able to access the system on basic hardware either simple desktops/laptops or tablets for flexibility on the customers end. Realistically they just need internet and browser support.

7. Assumptions and Dependencies (5 points)

Assumptions:

Restaurants already use Square POS and have API access enabled.

Internet connectivity is stable enough to support real-time synchronization.

Restaurant managers will input accurate ingredient and supplier data for correct inventory tracking.

Browsers used will support modern JavaScript features (ES6+) and secure HTTPS connections.

Dependencies:

Square API – If Square modifies its API or enforces stricter rate limits, functionality could be impacted.

Supabase – The system depends on Supabase for database services; downtime or API changes could disrupt operations.

Cloud Hosting Provider – Availability depends on third-party services.

Third-Party Libraries – Libraries such as AutoMapper, Swagger, and React must remain supported and compatible with framework updates.